

Project: Enterprise Asset Monitoring System (EAMS)

 Domain: Manufacturing/Energy

Objective

Create a backend system to:

- Register/manage factory assets (machines, equipment)
 - Simulate and store IoT sensor data (temperature, pressure, etc.)
 - Trigger alerts when sensor values cross thresholds
 - Maintain uptime/downtime logs
 - Enable role-based access for **Operators** and **Managers**
 - Schedule maintenance and track logs
-

Tech Stack (Backend)

Layer	Tech
Framework	Spring Boot
Security	Spring Security + JWT (Optional)
Persistence	Spring Data JPA
Database	MySQL
Build Tool	Maven / Gradle
Dev Tools	Spring Boot DevTools, Lombok (Optional)
Testing	JUnit
Optional	Swagger UI, ModelMapper

Roles & Access

Role	Access Level
Operator	View assigned assets, update asset status, log maintenance
Manager	Add/edit assets, assign assets, view reports, receive alerts

Key Modules

1. User & Role Management

- User registration/login
- Role assignment (Operator, Manager)
- JWT token generation & validation (**Optional**)

2. Asset Management

- CRUD on assets
- Assign assets to users
- Track asset location, type, and threshold limits

3. Sensor Data Ingestion (Simulation)

- Simulated API to push sensor data
- Store readings with timestamp
- Trigger alerts if data > threshold

4. Alert Management

- Create alerts based on rules
- Store active/resolved alerts
- Notification simulation (email/log console)

5. Maintenance & Uptime Logs

- Record scheduled maintenance
 - Auto-calculate uptime/downtime
 - Generate daily/weekly reports
-



Entity Design (ERD)

User

- |— id
- |— name
- |— email
- |— password
- |— role (ENUM: MANAGER, OPERATOR)

Asset

- |— id
- |— name
- |— type
- |— location
- |— thresholdTemp
- |— thresholdPressure
- |— assignedTo (User)

SensorData

- |— id
- |— assetId (FK)
- |— temperature
- |— pressure

|— timestamp

Alert

|— id
|— assetId (FK)
|— type (TEMP_HIGH / PRESSURE_HIGH)
|— message
|— status (ACTIVE / RESOLVED)
|— triggeredAt

MaintenanceLog

|— id
|— assetId (FK)
|— scheduledDate
|— completedDate
|— remarks

UptimeLog

|— id
|— assetId (FK)
|— startTime
|— endTime
|— status (UP / DOWN)

REST API Endpoints

AuthController

POST /api/auth/register

POST /api/auth/login

UserController (Manager only)

GET /api/users

PUT /api/users/{id}/role

AssetController

```
POST /api/assets           // Manager
GET  /api/assets           // Operator/Manager
GET  /api/assets/{id}
PUT  /api/assets/{id}
DELETE /api/assets/{id}
```

SensorDataController

```
POST /api/sensors/send-data // Simulated API
GET  /api/sensors/asset/{id}
```

AlertController

```
GET /api/alerts           // All alerts
PUT /api/alerts/{id}/resolve // Mark resolved
```

MaintenanceController

```
POST /api/maintenance
GET  /api/maintenance/asset/{id}
```

UptimeLogController

```
GET /api/uptime/asset/{id}
```

Spring Security + JWT

Auth Flow

- User logs in → JWT token issued (**Optional**)
- Token is added in `Authorization: Bearer <token>` header(**Optional**)
- Role-based access using `@PreAuthorize("hasRole('MANAGER')")` etc.

Spring Boot Configurations

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/asset_monitor
spring.datasource.username=postgres
spring.datasource.password=admin
spring.jpa.hibernate.ddl-auto=update
```

```
spring.security.jwt.secret=verysecretkey
spring.security.jwt.expiration=86400000
```

Sample Use Case

Scenario: A temperature sensor sends data **85°C** for Asset#101.

- If the threshold is **80°C** → System creates an **Alert**
 - Log is stored in **SensorData** + **Alert**
 - Operator is notified to schedule maintenance
-

Bonus Features to Add

Feature	How to Implement
WebSocket live dashboard	Use <code>@EnableWebSocketMessageBroker</code>
Email alerts	Use <code>JavaMailSender</code>
Report export	Use iText or Apache POI
Swagger Docs	Add Swagger using <code>springdoc-openapi-ui</code>

Scheduled jobs	Use @Scheduled for uptime calculations
----------------	--

Suggested Folder Structure

```
com.eams
├── config
├── controller
├── dto
├── entity
├── exception
├── repository
├── security
├── service
├── util
└── EamsApplication.java
```

Suggested Sprint Breakdown

Week	Deliverable
1	Setup project, implement user/login
2	Add asset management + CRUD
3	Simulate and store sensor data
4	Implement alert logic; view alert history
5	Maintenance + Uptime logging
6	Role-based dashboard view, testing & finalization